

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file           : main.c
 * @brief          : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */

#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include <stddef.h>           // Defines NULL pointer macro
#include "stm32f1xx_ll_usart.h" // FIX: Adds USART_InitTypeDef and UART functions
#include <string.h>           // Required for strcmp function operations

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */
#define BUTTON_PORT           GPIOB
#define BUTTON_PIN            LL_GPIO_PIN_12
#define BUTTON_CLK_PERIPH     LL_APB2_GRP1_PERIPH_GPIOB

// Define a structure to group all LED-related hardware parameters
typedef struct {
    GPIO_TypeDef *port; // Pointer to the GPIO Port (e.g., GPIOC)
    uint32_t pin;        // Specific GPIO Pin mask (e.g., LL_GPIO_PIN_13)
    uint8_t isActiveLow; // 1 if LED turns ON when pin is LOW (like Blue Pill)
} LED_TypeDef;

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

/* USER CODE BEGIN PV */

// Volatile tells compiler that this variable changes outside the main workflow (in ISR)
// __IO (volatile) is mandatory because this variable is modified inside an ISR
__IO uint8_t currentMode = MODE_LED_OFF; // Current operational mode controlled by EXTI
__IO uint8_t myData = 55U;               // A standard test variable
__IO uint8_t *myDataPointer = NULL;      // A pointer variable (initialized to NULL / address
0)

// 1. Define global static arrays for patterns (Stored in flash memory due to const)
// Flash-resident patterns to save RAM

```

```

static const uint16_t sosPattern[] = {
    150U, 150U, 150U, 150U, 150U, 500U, // Три точки и паузы (буква S)
    500U, 150U, 500U, 150U, 500U, 500U, // Три тире и паузы (буква O)
    150U, 150U, 150U, 150U, 150U, 800U // Три точки и финальная пауза перед повтором (буква S)
};

static const uint16_t heartBeatPattern[] = {100U, 200U, 100U, 800U};

// Create and initialize our onboard LED object using the new structure type
static const LED_TypeDef onboardLED =
{
    .port = GPIOC, // FIX: Added braces to satisfy GCC compiler rules
    .pin = LL_GPIO_PIN_13, // FIX: Added braces to satisfy GCC compiler rules
    .isActiveLow = 1U // Blue Pill onboard LED turns ON on LOW state
};

extern volatile char rx_buffer[];
extern volatile uint8_t rx_index;
extern volatile uint8_t command_ready;

/* USER CODE END PV */

/* Private function prototypes -----*/
/* USER CODE BEGIN PFP */
// Private Function Prototypes
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART1_UART_Init(void);

// STM32 peripheral function prototypes
uint32_t LL_GetTick(void);
void UART_Send_String(const char *str);
uint8_t Process_Button(void);
void Update_LED_Behavior(uint8_t mode);

// Now function accepts a pointer to our LED configuration and a pattern
void Play_Light_Pattern(const LED_TypeDef *led, const uint16_t *patternArray, uint8_t length,
uint8_t currentPatternMode);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

volatile uint32_t ms_ticks = 0; // Global millisecond counter

uint32_t LL_GetTick(void)
{
    return ms_ticks;
}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_AFIO);
    LL_APB1_GRP1_EnableClock(LL_APB1_GRP1_PERIPH_PWR);

```

```

/* System interrupt init*/
NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);

/* SysTick_IRQn interrupt configuration */
NVIC_SetPriority(SysTick_IRQn, NVIC_EncodePriority(NVIC_GetPriorityGrouping(), 15, 0));

/** NOJTAG: JTAG-DP Disabled and SW-DP Enabled
 */
LL_GPIO_AF_Remap_SWJ_NOJTAG();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

// Initialize all configured peripherals (including the EXTI button and LED GPIOs)
MX_GPIO_Init();
MX_USART1_UART_Init();

/* USER CODE BEGIN 2 */

LL_SYSTICK_EnableIT();

/* Enable USART1 Receive Data Register Not Empty interrupt */
LL_USART_EnableIT_RXNE(USART1);

UART_Send_String("STM32CubeIDE stm32-bluepill-edu has started\r\n");

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */

while (1)
{
    /* Handle asynchronous non-blocking LED state execution path */
    Update_LED_Behavior(currentMode);

    /* Check if a complete command line packet was flagged via the ISR */
    if (command_ready)
    {
        command_ready = 0;

        /* Parse and evaluate the validated text string token */
        if (strcmp((const char*)rx_buffer, "OFF") == 0)
        {
            currentMode = MODE_LED_OFF;
            UART_Send_String("\r\n-> Executed: LED turned OFF\r\n");
        }
        else if (strcmp((const char*)rx_buffer, "ON") == 0)
        {
            currentMode = MODE_LED_ON;
            UART_Send_String("\r\n-> Executed: LED turned ON\r\n");
        }
        else if (strcmp((const char*)rx_buffer, "SOS") == 0)
        {
            currentMode = MODE_LED_SOS;
            UART_Send_String("\r\n-> Executed: Mode SOS activated\r\n");
        }
        else if (strcmp((const char*)rx_buffer, "HB") == 0)
        {
            currentMode = MODE_LED_HTB;
            UART_Send_String("\r\n-> Executed: Mode HeartBit activated\r\n");
        }
        else

```

```

    {
        UART_Send_String("\r\n-> Error: Unknown command packet string!\r\n");
    }

    __disable_irq();

    /* CRITICAL: Safely flush the buffer memory before enabling the next packet reception */
    memset((void*)rx_buffer, 0, RX_BUF_SIZE);
    rx_index = 0;

    __enable_irq();

}

/* 1ms breath time to prevent debug locking issues */
LL_mDelay(1);
}

/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    LL_FLASH_SetLatency(LL_FLASH_LATENCY_0);
    while(LL_FLASH_GetLatency() != LL_FLASH_LATENCY_0)
    {
    }
    LL_RCC_HSI_SetCalibTrimming(16);
    LL_RCC_HSI_Enable();

    /* Wait till HSI is ready */
    while(LL_RCC_HSI_IsReady() != 1)
    {
    }
    LL_RCC_SetAHBPrescaler(LL_RCC_SYSCLK_DIV_1);
    LL_RCC_SetAPB1Prescaler(LL_RCC_APB1_DIV_1);
    LL_RCC_SetAPB2Prescaler(LL_RCC_APB2_DIV_1);
    LL_RCC_SetSysClkSource(LL_RCC_SYS_CLKSOURCE_HSI);

    /* Wait till System clock is ready */
    while(LL_RCC_GetSysClkSource() != LL_RCC_SYS_CLKSOURCE_STATUS_HSI)
    {
    }
    LL_Init1msTick(8000000);
    LL_SetSystemCoreClock(8000000);
}

/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{
    /* USER CODE BEGIN USART1_Init 0 */

    LL_GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* Peripheral clock enable */

    /* 1. Enable Peripheral Clocks for GPIOA and USART1 */
    LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_USART1);

```

```

LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOA);

/* USER CODE END USART1_Init 0 */

/* USER CODE BEGIN USART1_Init 1 */

/**USART1 GPIO Configuration
PA9  -----> USART1_TX
PA10 -----> USART1_RX
*/

GPIO_InitStruct.Pin          = LL_GPIO_PIN_9;
GPIO_InitStruct.Mode         = LL_GPIO_MODE_ALTERNATE;
GPIO_InitStruct.Speed        = LL_GPIO_SPEED_FREQ_HIGH;
GPIO_InitStruct.OutputType   = LL_GPIO_OUTPUT_PUSHPULL;
LL_GPIO_Init(GPIOA, &GPIO_InitStruct);

GPIO_InitStruct.Pin = LL_GPIO_PIN_10;
GPIO_InitStruct.Mode = LL_GPIO_MODE_INPUT;          // Configure PA10 as Input
LL_GPIO_SetPinPull(GPIOA, LL_GPIO_PIN_10, LL_GPIO_PULL_UP); // Enable Pull-up resistor
LL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/* USER CODE END USART1_Init 1 */

/* USER CODE BEGIN USART1_Init 2 */

/* Configure USART1 Hardware Parameters */
// We assume default 8MHz or 72MHz clock.
// LL_USART_Init calculates the baudrate automatically.
LL_USART_InitTypeDef USART_InitStruct = {0};
USART_InitStruct.BaudRate              = 115200U;          // Speed: 115200 bits per
second
USART_InitStruct.DataWidth              = LL_USART_DATAWIDTH_8B; // 8 bits of data
USART_InitStruct.StopBits               = LL_USART_STOPBITS_1;  // 1 stop bit
USART_InitStruct.Parity                 = LL_USART_PARITY_NONE;  // No parity checking
USART_InitStruct.TransferDirection      = LL_USART_DIRECTION_TX_RX; // Enable both Transmit
and Receive
USART_InitStruct.HardwareFlowControl    = LL_USART_HWCONTROL_NONE; // No flow control
USART_InitStruct.OverSampling           = LL_USART_OVERSAMPLING_16;
LL_USART_Init(USART1, &USART_InitStruct);
LL_USART_ConfigAsyncMode(USART1);
LL_USART_Enable(USART1);

/* REQUIRED: Enable Receive Data Register Not Empty interrupt */
LL_USART_EnableIT_RXNE(USART1);

/* CRITICAL: Enable USART RX interrupt */
LL_USART_EnableIT_RXNE(USART1);

/* Enable USART1 NVIC Interrupt*/
NVIC_SetPriority(USART1_IRQn, NVIC_EncodePriority(NVIC_GetPriorityGrouping(), 5, 0));
NVIC_EnableIRQ(USART1_IRQn);

/* USER CODE END USART1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    LL_GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOA);

```

```

LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOB);
LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOC);
LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_AFIO);

/* USER CODE END MX_GPIO_Init_1 */

GPIO_InitStruct.Pin          = LL_GPIO_PIN_13;
GPIO_InitStruct.Mode         = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed        = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType   = LL_GPIO_OUTPUT_PUSHPULL;
LL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */

/* Enable Clocks for AFIO (AFIO is required for EXTI mapping on F1 series) */
LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_AFIO);

LL_GPIO_ResetOutputPin(GPIOC, LL_GPIO_PIN_13);
LL_GPIO_SetPinMode(GPIOC, LL_GPIO_PIN_13, LL_GPIO_MODE_OUTPUT);

/* Configure PB12 pin as Input with Pull-up resistor */
LL_GPIO_SetPinMode(BUTTON_PORT, BUTTON_PIN, LL_GPIO_MODE_INPUT);
LL_GPIO_SetPinPull(BUTTON_PORT, BUTTON_PIN, LL_GPIO_PULL_UP);

/* Map EXTI Line 12 to GPIOB Port (Connects PB12 to EXTI12 controller) */
LL_GPIO_AF_SetEXTISource(LL_GPIO_AF_EXTI_PORTB, LL_GPIO_AF_EXTI_LINE12);

LL_EXTI_EnableIT_0_31(LL_EXTI_LINE_12);
LL_EXTI_EnableFallingTrig_0_31(LL_EXTI_LINE_12);

/* Enable EXTI Line 12 Interrupt in NVIC controller */
// Line 12 on STM32F1 belongs to a shared vector EXTI15_10_IRQn (Lines 10 to 15)
NVIC_SetPriority(EXTI15_10_IRQn, NVIC_EncodePriority(NVIC_GetPriorityGrouping(), 5, 0));
NVIC_EnableIRQ(EXTI15_10_IRQn);

/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

void Update_LED_Behavior(uint8_t mode)
{
    switch (mode)
    {
        case MODE_LED_OFF: /* Mode 0: LED OFF */
            // High level turns off Active LOW LED
            LL_GPIO_SetOutputPin(onboardLED.port, onboardLED.pin);
            break;

        case MODE_LED_ON: /* Mode 1: LED ON */
            // Low level turns on Active LOW LED
            LL_GPIO_ResetOutputPin(onboardLED.port, onboardLED.pin);
            break;

        case MODE_LED_SOS: /* Mode 2: SOS Morse Code (. . . - - - . . .) */
            Play_Light_Pattern(&onboardLED, sosPattern, sizeof(sosPattern)/sizeof(sosPattern[0]),
                               currentMode);
            break;

        case MODE_LED_HTB:
            Play_Light_Pattern(&onboardLED, heartBeatPattern, sizeof(heartBeatPattern)/sizeof(
            heartBeatPattern[0]), currentMode);
            break;

        default:
            // Safe-state: Turn off processing when MODE_LED_OFF is selected
            Play_Light_Pattern(&onboardLED, NULL, 0, MODE_LED_OFF);
            break;
    }
}
}

```

```

/**
 * @brief Plays a light pattern on a specific LED defined by a structure pointer.
 * @param led: Pointer to the LED structural configuration.
 * @param patternArray: Pointer to the intervals array.
 * @param length: Number of elements in the array.
 */
void Play_Light_Pattern(const LED_TypeDef *led, const uint16_t *patternArray, uint8_t length,
uint8_t currentPatternMode)
{
    // Persist timing and step state between sequential function calls
    static uint32_t last_led_update = 0;
    static uint8_t pattern_step = 0U;

    // Tracks the operation mode from the PREVIOUS function execution
    static uint8_t last_mode = 0U;

    // 1. OPERATION MODE CHANGE DETECTION
    if (currentPatternMode != last_mode)
    {
        /*
        // Buffer where the string will be written (allocated with extra safety margin)
        char tx_buffer[64];
        // Format the string: %d specifies a signed decimal integer
        snprintf(tx_buffer, sizeof(tx_buffer), "@Play_Light_Pattern : length = %d\r\n",
length);
        // Transmit the formatted string over UART
        UART_Send_String(tx_buffer);

        // Format the string: %d specifies a signed decimal integer
        snprintf(tx_buffer, sizeof(tx_buffer), "@Play_Light_Pattern : currentPatternMode =
%d\r\n", currentPatternMode);
        // Transmit the formatted string over UART
        UART_Send_String(tx_buffer);

        */

        pattern_step = 0U; // Reset step index to the beginning of the new pattern
        last_led_update = LL_GetTick(); // Reset time reference to the current tick
        last_mode = currentPatternMode; // Update the tracking variable with the new mode

        // Optional: Turn off the LED during transition to start with a clean state
        // High level turns off Active LOW LED
        LL_GPIO_SetOutputPin(led->port, led->pin);
    }

    // Immediately exit if the system is configured to "Off"
    if (currentPatternMode == MODE_LED_OFF) {
        return;
    }

    // Safe-check: Guard against null pointers or uninitialized zero-length arrays
    if (patternArray == NULL || length == 0U) {
        return;
    }

    // 2. BOUNDARY SANITY CHECK
    if (pattern_step >= length) {
        pattern_step = 0U; // Loop the pattern seamlessly back to the start
    }

    // 3. NON-BLOCKING ASYNCHRONOUS TIMER (Replaces the traditional for-loop)
    // The uint32_t subtraction correctly handles LL_GetTick rollover every 49.7 days
    // 3. CORRECT ASYNCHRONOUS TIMER
    // Extract the time interval: for steps 0 and 1 it uses element, for 2 and 3 - element [1]
    uint16_t current_interval = patternArray[pattern_step];

    if ((LL_GetTick() - last_led_update) >= current_interval)

```

```

{
    last_led_update = LL_GetTick(); // Reset the timer
    // State management instead of Toggle
    // If the step is even (0, 2, 4...) -> TURN ON the LED
    if (pattern_step % 2 == 0) {
        LL_GPIO_ResetOutputPin(led->port, led->pin); // Low = ON for Active LOW
    }
    // If the step is odd (1, 3, 5...) -> TURN OFF the LED
    else {
        LL_GPIO_SetOutputPin(led->port, led->pin); // High = OFF for Active LOW
    }
    pattern_step++; // Move to the next step of the phase
}
}

/**
 * @brief Sends a null-terminated string over USART1 using pointers.
 * @param str: Pointer to the character array (string).
 * @retval None
 */
void UART_Send_String(const char *str)
{
    // Loop runs until the pointer hits the '\0' null-terminator byte
    while (*str != '\0')
    {
        // 1. Wait until the Transmit Data Register is empty (ready to accept next byte)
        while (LL_USART_IsActiveFlag_TXE(USART1) == 0U)
        {
            __NOP(); // Wait
        }

        // 2. Transmit the current single character by dereferencing the pointer
        LL_USART_TransmitData8(USART1, *str);

        // 3. Move the pointer to the next character in memory (Pointer arithmetic)
        str++;
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

```



```
}  
#endif /* USE_FULL_ASSERT */
```